Data Analysis

Classification

Section 4

Eng. Asmaa Fathy

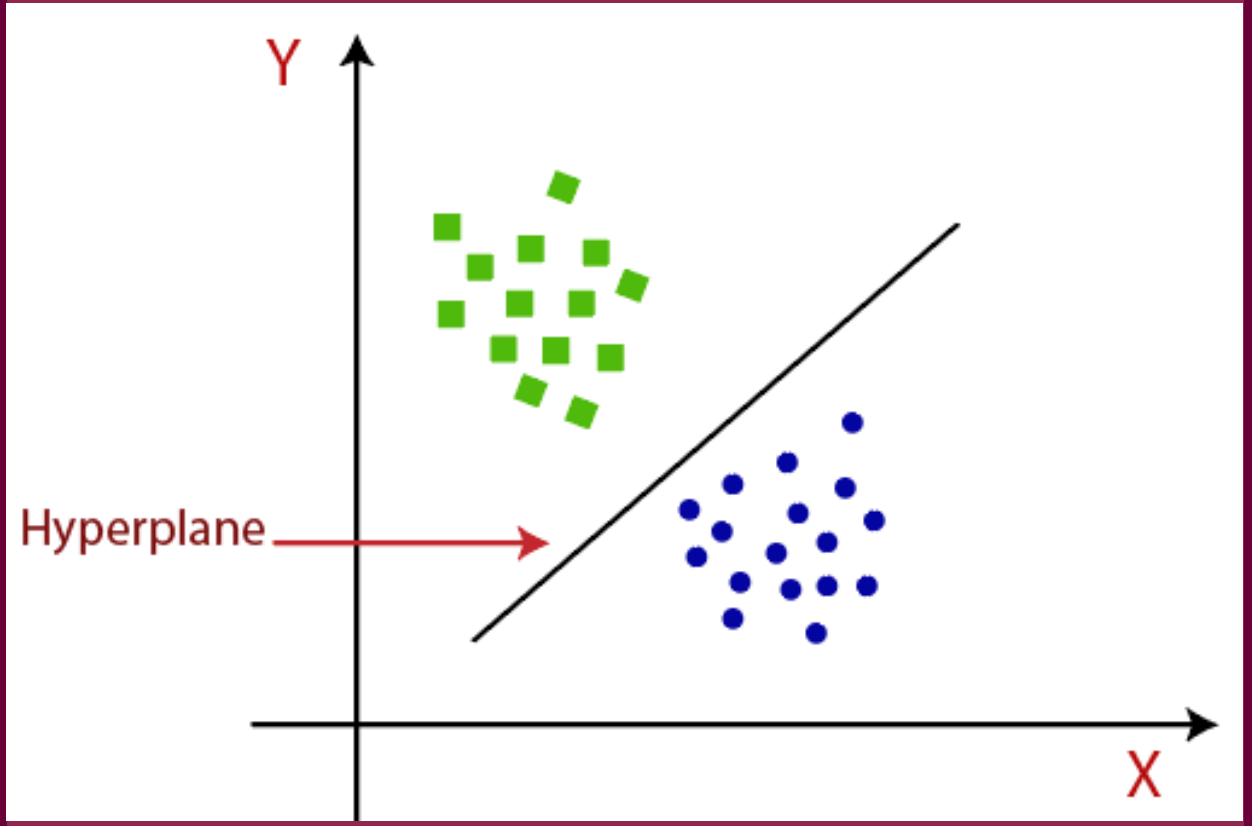
Classification

- NEAREST NEIGHBOR CLASSIFIERS
- DECISION TREE CLASSIFIERS
- SUPPORT VECTOR MACHINE CLASSIFIERS
- NAIVE BAYES CLASSIFIERS
- LOGISTIC REGRESSION CLASSIFIERS

SUPPORT VECTOR MACHINE CLASSIFIERS

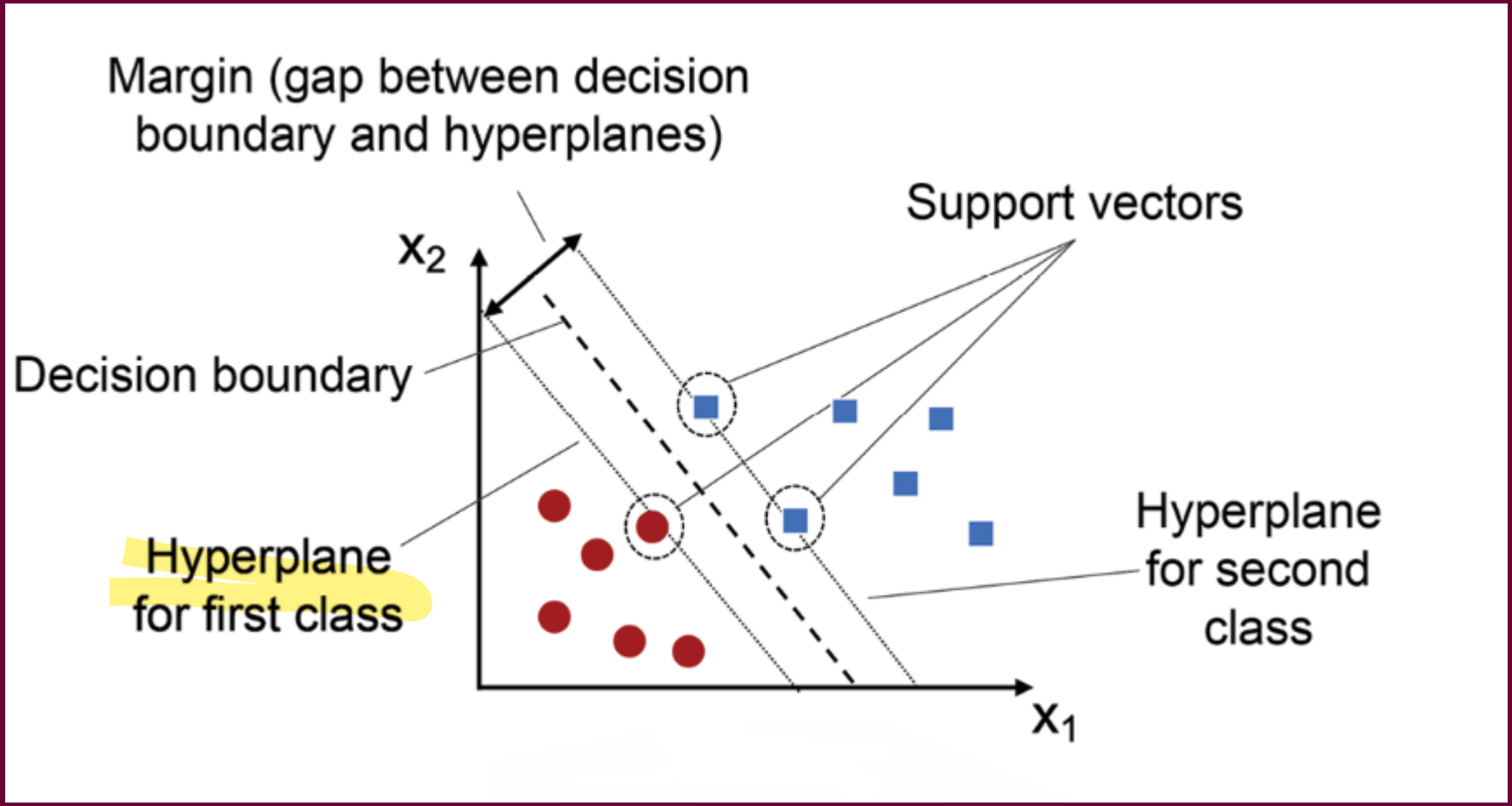
- Support Vector Machines (SVMs) are powerful and versatile machine learning algorithms used for classification and regression tasks.
- Support Vector Machine Classifiers (SVMs) are supervised learning algorithms that excel in both linear and non-linear classification tasks. They work by finding the optimal hyperplane that best separates data into distinct classes.

SVM tries to find the best boundary that separates the classes.



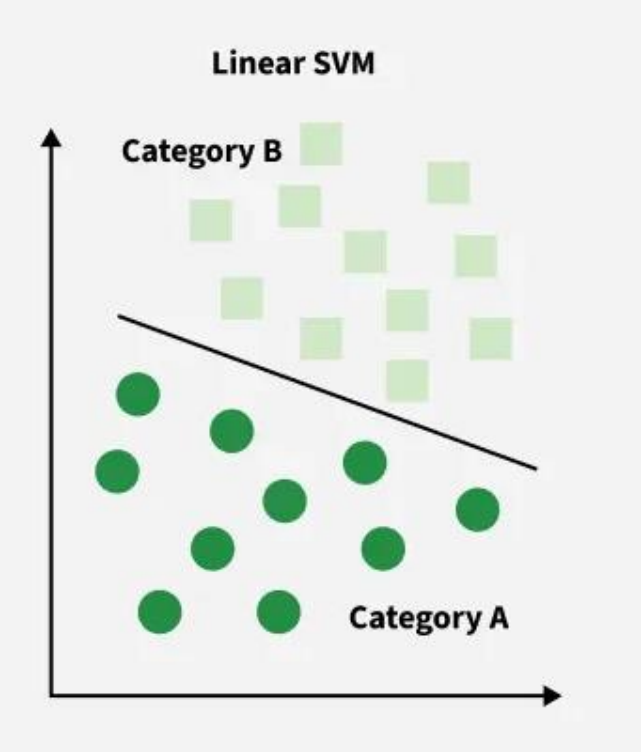
The boundary that separates classes is called a Hyperplane.

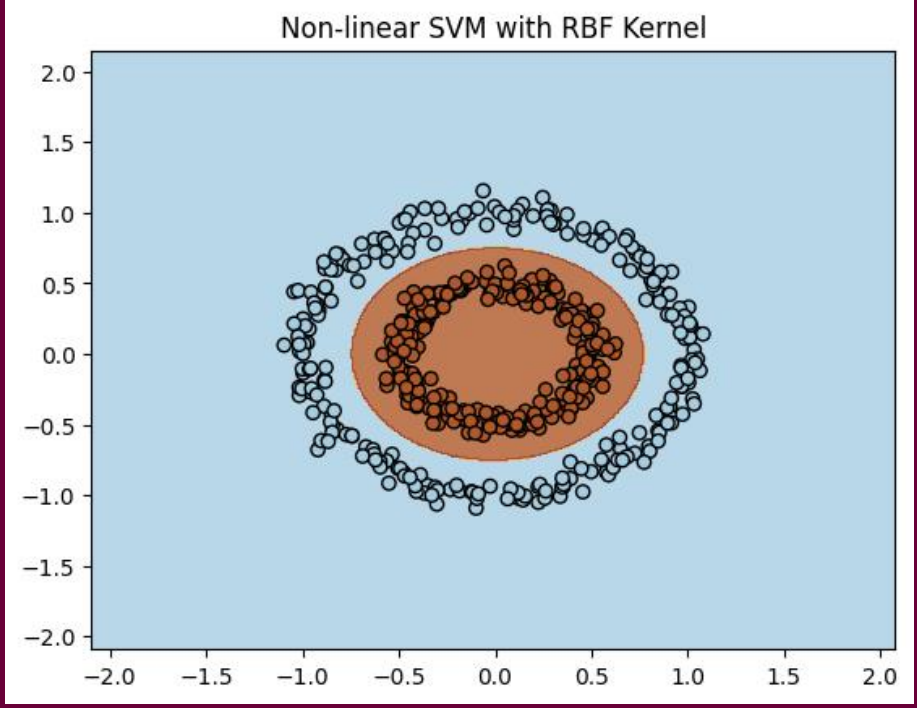
- SVM chooses the boundary with the maximum margin.
- Margin = distance between the boundary and the nearest points.
- The closest data points to the boundary are called support vectors.
- These points determine the position of the hyperplane.

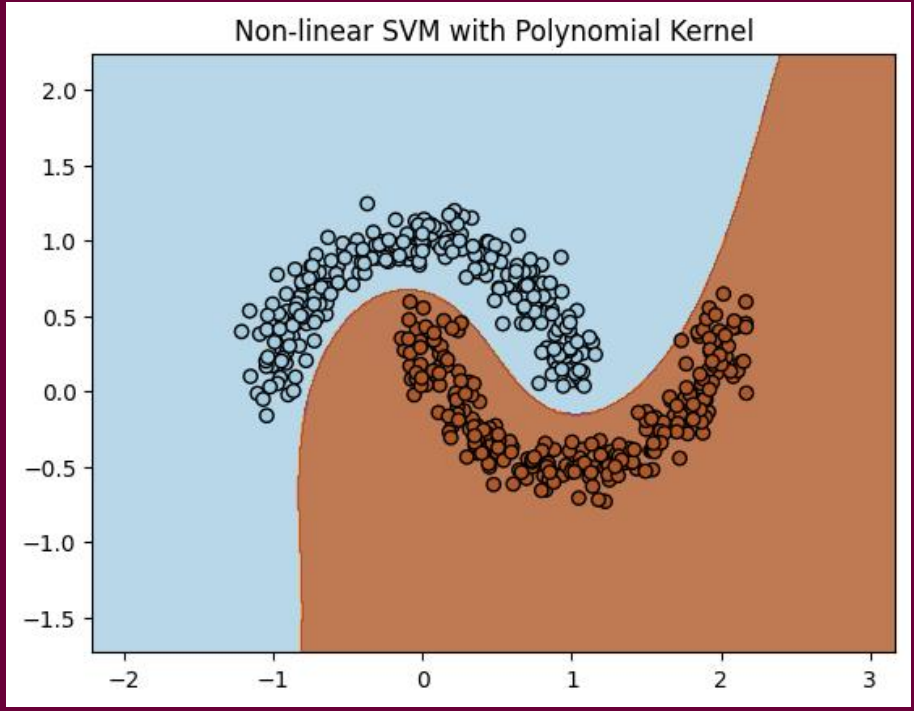


Kernel Trick

- Sometimes data cannot be separated using a straight line.
- SVM uses Kernel functions to transform data to a higher dimension.
- Types of Kernels:







Iris Binary Classification Using SVM

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
df = pd.DataFrame({'Sepal length': iris.data[:,0],
'Sepal width': iris.data[:,1],
'Petal length':iris.data[:,2],
'Petal width':iris.data[:,3],
'Species':iris.target})
df.head()
```

	Sepal length	Sepal width	Petal length	Petal width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

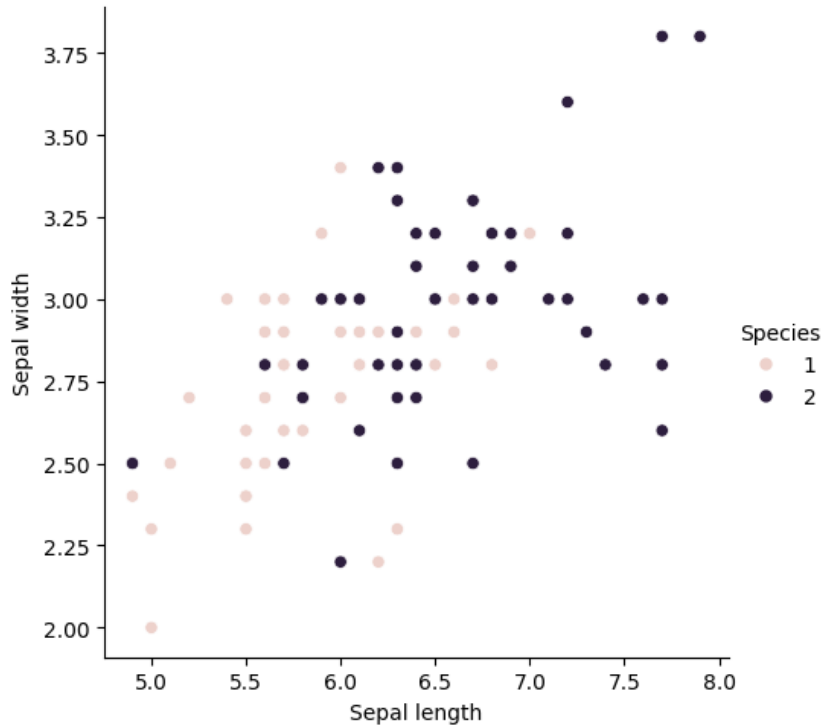
```
df = df[df['Species'] !=0]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100 entries, 50 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Sepal length  100 non-null    float64
 1   Sepal width   100 non-null    float64
 2   Petal length  100 non-null    float64
 3   Petal width   100 non-null    float64
 4   Species       100 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 4.7 KB
```

```
sns.relplot(data = df, x = 'Sepal length',y= 'Sepal width', hue = 'Species')
```

```
<seaborn.axisgrid.FacetGrid at 0x21c52b1bb60>
```



Iris Binary Classification Using SVM

```
from sklearn.model_selection import train_test_split
X = df[df.columns[:2]]
y = df[df.columns[-1]]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20)
```

```
X_train[:5]
```

	Sepal length	Sepal width
122	7.7	2.8
115	6.4	3.2
136	6.3	3.4
59	5.2	2.7
55	5.7	2.8

```
from sklearn import svm
model = svm.SVC(kernel='linear')
classifier = model.fit(X_train, y_train)
```

```
y_pred = classifier.predict(X_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(accuracy_score(y_test,y_pred))
```

```
[[10  1]
 [ 0  9]]
              precision    recall  f1-score   support

           1       1.00      0.91      0.95        11
           2       0.90      1.00      0.95         9

    accuracy                           0.95        20
   macro avg       0.95      0.95      0.95        20
weighted avg       0.96      0.95      0.95        20

0.95
```

Naïve Bayes Classifier

- Naïve Bayes is a machine learning model that uses Bayes' theorem to classify data.
- It is called 'Naïve' because it assumes that features are independent of each other, an assumption that may not be entirely true but makes the calculations easier.
- **How it works:**
- The probability of each class is calculated based on the input using Bayes' theorem.
- Selects the category with the highest probability based on the data.

Bayes' Theorem: Basics

- Bayes' Theorem is a mathematical rule used to determine the probability of an event occurring based on prior information associated with it.
- The basic formula is:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- P(A|B): Probability of the event A if B has happened.
- P(B|A): Probability of the event B if A has happened.
- P(A): prior probability of the event A
- P(B): prior probability of the event B.

Example: Disease Diagnosis using Naïve Bayes Classifier

- **Scenario :** We assume that you are a doctor and want to predict whether a patient will develop a disease based on the symptoms he or she reports. We have a dataset that shows the relationship between symptoms and disease diagnosis.
- **Question:** If a patient reports Fever = Yes, Cough = Yes, and Fatigue = No, is it likely that he or she has the disease?

Dataset

Symptom 1 (Fever)	Symptom 2 (Cough)	Symptom 3 (Fatigue)	Disease (Yes/No)
Yes	Yes	Yes	Yes
Yes	Yes	No	Yes
No	Yes	Yes	No
Yes	No	Yes	Yes
No	No	No	No
No	Yes	No	No
Yes	Yes	Yes	Yes
Yes	No	No	No

1. Calculate Prior Probabilities:

P(Disease=Yes) = Number of cases “Yes”/ Total cases = 4/8 = 0.5
P(Disease=No) = Number of cases “No”/ Total cases = 4/8 = 0.5

2. Calculate Conditional Probabilities:

- P(Fever=Yes|Disease=Yes) =
  (Number of Fever = Yes * Number of Disease = Yes) / (Number of Disease = Yes) = 4/4 = 1.0
- P(Cough=Yes|Disease=Yes) =
  (Number of Cough = Yes * Number of Disease = Yes) / (Number of Disease = Yes) = ¾ = 0.75
- P(Fatigue=No|Disease=Yes) =
  (Number of Fatigue = No * Number of Disease = Yes) / (Number of Disease = Yes) = ¼ = 0.25
- P(Fever=Yes|Disease=No) =
  (Number of Fever = Yes * Number of Disease = No) / (Number of Disease = No) = ¼ = 0.25
- P(Cough=Yes|Disease=No) =
  (Number of Cough = Yes * Number of Disease = No) / (Number of Disease = No) = 2/4 = 0.5
- P(Fatigue=No|Disease=No)=
  (Number of Fatigue = No * Number of Disease = No) / (Number of Disease = No) = ¾ = 0.75

3. Apply Naïve Bayes Formula:

A. **Calculate $P(Disease=Yes|Symptoms)$:**

- $P(Disease=Yes|Symptoms)=$ $P(Fever=Yes|Disease=Yes)\cdot P(Cough=Yes|Disease=Yes)\cdot P(Fatigue=No|Disease=Yes)\cdot P(Disease=Yes)$ = 1.0 · 0.75 · 0.25 · 0.5 = 0.09375

B. **Calculate $P(Disease=No|Symptoms)$:**

- P(Disease=No|Symptoms)= P(Fever=Yes|Disease=No)·P(Cough=Yes|Disease=No)·P(Fatigue=No|Disease=No)·P(Disease=No) = 0.25 · 0.5 · 0.75 · 0.5 = 0.0469

3. Compare Probabilities:

- **P(Disease=Yes|Symptoms) = 0.09375**
- **$P(Disease=No|Symptoms)$ = 0.0469**
- **Since $P(Disease=Yes|Symptoms) > P(Disease=No|Symptoms)$, The patient likely has the disease.**

Iris Binary Classification Using Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
result = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(result)
result1 = classification_report(y_test, y_pred)
print("Classification Report:",)
print(result1)
result2=accuracy_score(y_test,y_pred)
print("Accuracy:",result2)
```

```
Confusion Matrix:
[[10  1]
 [ 1  8]]
Classification Report:
              precision    recall  f1-score   support

           1       0.91      0.91      0.91        11
           2       0.89      0.89      0.89         9

    accuracy                           0.90        20
   macro avg       0.90      0.90      0.90        20
weighted avg       0.90      0.90      0.90        20

Accuracy: 0.9
```

LOGISTIC REGRESSION CLASSIFIERS

- Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values, it predicts the probability that an input belongs to a specific class.
- It is used for binary classification where the output can be one of two possible categories, such as Yes/No, True/False, or 0/1.
- It uses a sigmoid function to convert inputs into a probability value between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Input	Output
Large Positive	Close to 1
Large Negative	Close to 0

Example:
If output = 0.8 → Probability = 80% → Pass
If output = 0.2 → Probability = 20% → Fail

- **We use a threshold to decide the class**
- **Usually: `Threshold = 0.5`**

Probability	Class
≥ 0.5	1 (Pass)
< 0.5	0 (Fail)

If probability is high → classify as Pass
If probability is low → classify as Fail

Types of Logistic Regression

Binomial

Two classes (0 or 1)

Multinomial

More than two unordered classes (cat, dog, sheep)

Ordinal

More than two ordered classes (low, medium, high)

Iris Binary Classification Using Logistic Regression

```
from sklearn.linear_model import LogisticRegression
classifier=LogisticRegression().fit(X,y)
classifier.fit(X_train,y_train)
```

▾ LogisticRegression

▶ Parameters

```
y_pred=classifier.predict(X_test)
```

```
from sklearn.metrics import classification_report,confusion_matrix,accuracy_sco
result=confusion_matrix(y_test,y_pred)
print("ConfusionMatrix:")
print(result)
report=classification_report(y_test,y_pred)
print("ClassificationReport:",)
print(report)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",accuracy)
```

```
ConfusionMatrix:
[[10  1]
 [ 0  9]]
ClassificationReport:
              precision    recall  f1-score   support

           1       1.00      0.91      0.95        11
           2       0.90      1.00      0.95         9

    accuracy                           0.95        20
   macro avg       0.95      0.95      0.95        20
weighted avg       0.96      0.95      0.95        20

Accuracy: 0.95
```

Classifier Evaluation Metrics

- When evaluating the performance of a classification model, we rely on several metrics that assess its accuracy, reliability, and ability to distinguish between classes. Below is a detailed explanation of each metric using a real-world example: Email Spam Classification.

Confusion Matrix (Truth Table)

Before diving into metrics, let's define a confusion matrix, which is the foundation of all evaluation metrics.
TP (True Positive): Emails correctly classified as spam.
FN (False Negative): Spam emails wrongly classified as not spam.
FP (False Positive): Non-spam emails wrongly classified as spam.
TN (True Negative): Non-spam emails correctly classified as not spam.

	Predicted Spam	Predicted Not Spam
Actual Spam	True Positive (TP)	False Negative (FN)
Actual Not Spam	False Positive (FP)	True Negative (TN)

1. Accuracy

- The proportion of correctly classified emails (spam or not spam) out of all emails.
- **Accuracy = (TP + TN)/All**
- Real-world Example:
  - Total Emails: 1000
  - Spam Emails: 200
  - Non-Spam Emails: 800
  - Model Results: TP = 150, TN = 700, FP = 100, FN = 50
  - **Accuracy = (150 + 700)/1000**

2. Error Rate

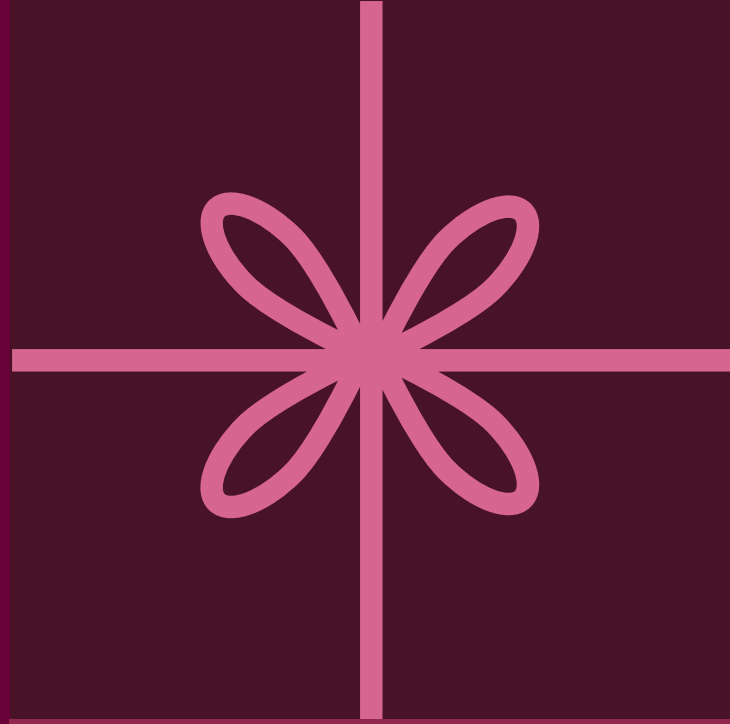
- The proportion of incorrectly classified emails.
- **Error rate: 1 – accuracy, or**
- **Error rate = (FP + FN)/All**
- **From Previous Example :**
- Model Results: TP = 150, TN = 700, FP = 100, FN = 50
- **Error Rate=1−0.85=0.15(15%)**
- **Or Error Rate = (100+50)/1000 = 0.15 (15%)**

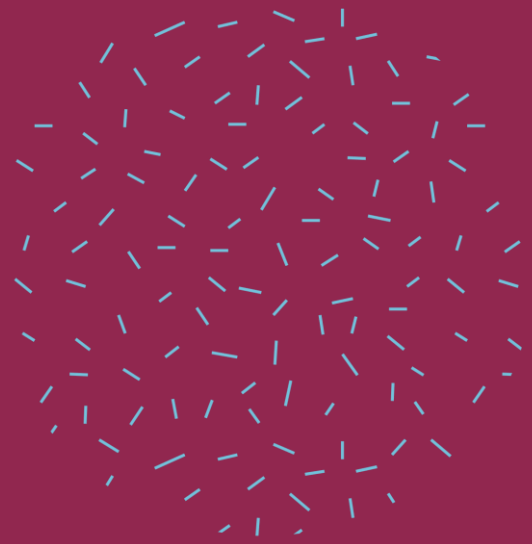
3. Sensitivity (Recall)

- The ability of the model to correctly identify spam emails.
- **Sensitivity** (Recall) **= TP/P = TP/(TP+FN)**
- **From Previous Example :**
- Model Results: TP = 150, TN = 700, FP = 100, FN = 50
- **Sensitivity** (Recall) **= TP/P = TP/(TP+FN) = 150/(150+50) = 0.75 (75%).**
- This means the model correctly identifies 75% of spam emails.

4. Specificity

- The ability of the model to correctly identify non-spam emails.
- **Specificity = TN/N = TN/(TN+FP)**
- From Previous Example :
- Model Results: TP = 150, TN = 700, FP = 100, FN = 50
- **Specificity = TN/N = TN/(TN+FP) = 700/(700+100) = 0.875 (87.5%).**
- This means the model correctly identifies 87.5% of non-spam emails.



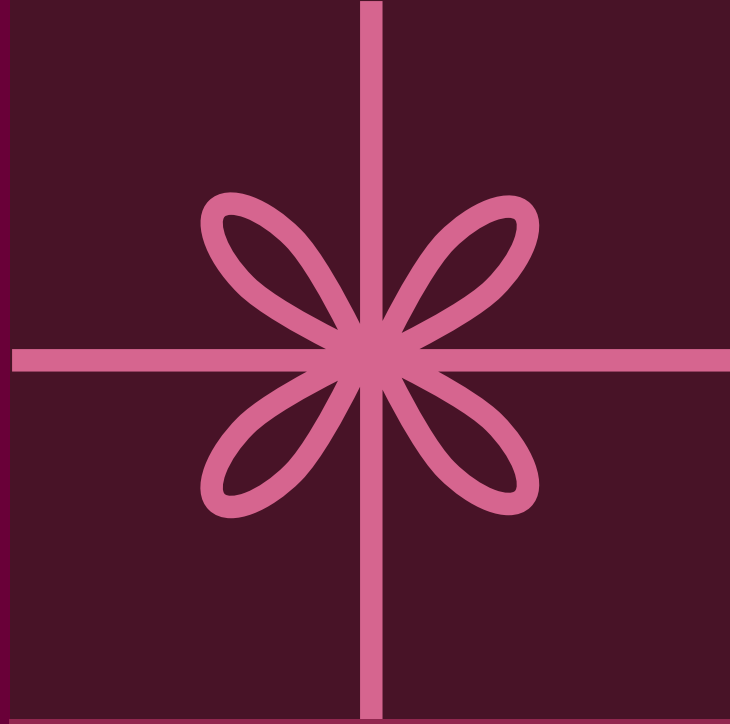


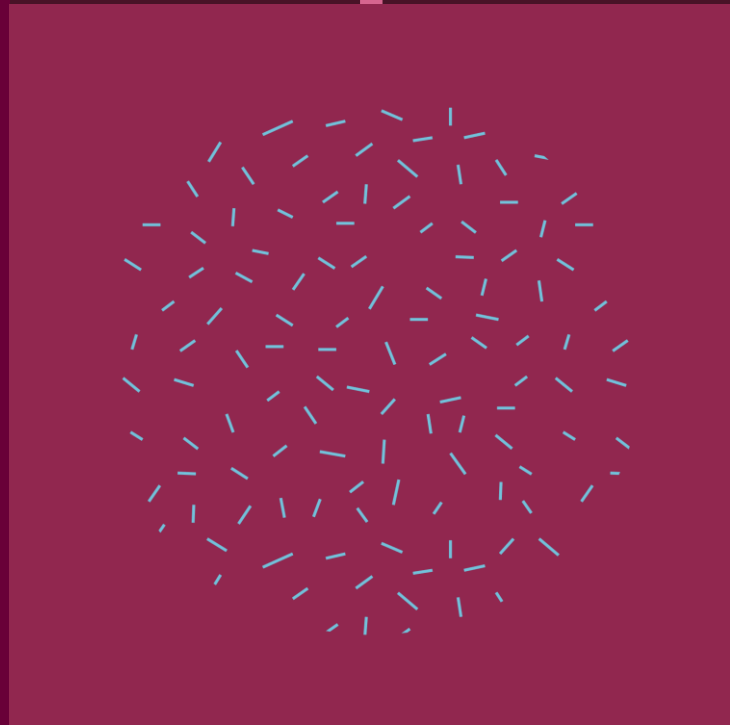
5. Precision

- The proportion of emails predicted as spam that are actually spam.
- **P =Precision = TP/(TP+FP)**
- From Previous Example :
- Model Results: TP = 150, TN = 700, FP = 100, FN = 50
- **P =Precision = TP/(TP+FP) = 150/(150+100) = 0.6 (60%)**
- This means 60% of emails classified as spam are actually spam.

6. F1-Score

- A metric that balances Precision and Recall, especially useful for imbalanced datasets.
- **F1 = 2P * R / (P + R)**
- From Previous Example :
- P (Precision) = **0.6 (60%)**
- R (Recall) = **0.75 (75%).**
- **F1 = 2 * 0.6 * 0.75 / (0.6 + 0.75) = 0.666(66.6%)**





Exercise :

- TP = 90 , FN = 210 , FP = 140 , TN = 9560

Actual Class\Predicted class	cancer = yes	cancer = no	Total
cancer = yes	**90**	**210**	300
cancer = no	**140**	**9560**	9700
Total	230	9770	10000

Solution

- **TP = 90 , FN = 210 , FP = 140 , TN = 9560**
- **Accuracy = (TP + TN)/All = (90 + 9560)/10000 = 0.965 (96.5%)**
- **Error rate: 1 – accuracy = 1 – 0.965 = 0.035 (3.5%)**
- **Sensitivity (Recall) = TP/P = TP/(TP+FN) = 90/(90+210) = 0.3 (30%)**
- **Specificity = TN/N = TN/(TN+FP) = 9560/(9560+140) = 0.9855 (98.56%)**
- **P =Precision = TP/(TP+FP) = 90/(90+140) = 0.3913 (39.13%)**
- **F1 = 2P * R / (P + R) = 2 * 39.13% * 30% / (39.13% + 30%) =** 33.96%

Task

- Classification of the three flower types in Iris Dataset using SVM Python.
- Classification of the three flower types in Iris Dataset using Naive Bayes Python.
- Classification of the three flower types in Iris Dataset using Logistic Regression Python.

- **Scenario:** We assume that you are a botanist and want to predict the type of an iris flower based on its features.
- We have: Sepal Length (Long / Short) , Petal Length (Long / Short)
- Classes: Setosa and Versicolor

Sepal	Petal	Class
Long	Short	Setosa
Long	Short	Setosa
Short	Short	Setosa
Long	Long	Versicolor
Short	Long	Versicolor
Long	Long	Versicolor
Short	Long	Versicolor
Long	Short	**Versicolor**

Question:
If a flower has:
Sepal Length = Long
Petal Length = Short
Is it **Setosa or Versicolor?**

- **You have an email classification system to identify messages as spam or not spam. After testing the system on 2000 messages, the results were as follows:**

	Predicted Spam	Predicted Not Spam
Actual Spam	150	50
Actual Not Spam	200	1600

Calculate each of the following:

- **Overall Accuracy**
- **Error Rate**
- **Sensitivity/Recall**
- **Specificity**
- **Precision**
- **F1 Score**

Thank You